Assignment-2: Software Testing and Quality Assurance

MCA | Complete answers to all 15 questions from the uploaded assignment sheet

How to use: Read the concept, study the steps/code, and run each Java program in a separate file named after its public class. Outputs are representative; thread output order may vary.

1. Testing strategy for an Online Examination Management System

Testing fundamentals: testing reveals defects, is context-dependent, should start early, and cannot prove the absence of all defects. Testing is risk-based because payments, exam submission, and result accuracy are high-impact features.

Stages and activities: (1) Requirements stage—review requirements, acceptance criteria, payment rules, eligibility, privacy and result calculations (static testing; verification). (2) Design stage—review workflows, database schema, security design and testability; create test plan and traceability matrix. (3) Unit testing—test registration validation, subject selection, fee calculation, admit-card eligibility and result calculation. (4) Integration testing—test student portal with payment gateway, database, notification and result services. (5) System testing—end-to-end scenarios across browsers/devices, performance, security, usability, accessibility, recovery and compatibility. (6) Acceptance testing—students/admins verify realistic workflows against business needs (validation). (7) Regression testing—rerun critical tests after fixes/releases.

Static vs dynamic: static testing reviews requirements, design, code and test cases without executing the program; dynamic testing executes the software with test data. Verification asks “Are we building the product right?”; validation asks “Are we building the right product?” Each activity is required to find defects early, reduce rework, verify integrations, protect payments/data, and confirm the system meets user needs.

2. Static testing defects in online banking

Reviews can detect unclear requirements for transfer limits, fees, cut-off times, failed transfers and reversals; missing acceptance criteria; contradictory rules; insecure design (weak session expiry or authorization); SQL injection risks in query construction; hard-coded credentials; incorrect currency/rounding logic; race conditions in balance updates; duplicated transfer logic; inadequate exception handling/logging; missing audit trails; and code that exposes account data.

Use requirement reviews, threat modelling, design walkthroughs, code inspection, static analysis, linting and dependency/security scans. Record each defect with location, severity, evidence and owner. Fixing a requirement or code defect before execution is usually cheaper than diagnosing it after integration, when it may involve test data, deployment, logs, and multiple components. Static testing complements—not replaces—dynamic testing.

3. Formal review process for a 2,000-line module

Participants: moderator, author, reviewer(s) with domain knowledge, recorder/scribe, and optionally QA/security representative. Preparation: define scope and entry criteria; share code, standards and checklist in advance; author runs basic checks; reviewers inspect independently and log defects.

Stages: planning → overview/kick-off → individual preparation → review meeting (focus on finding defects, not blaming the author) → rework → follow-up/closure. Classify defects as critical/high/medium/low and by type: naming/style, duplication, complexity, documentation, logic, security, performance, maintainability. Checklist: naming consistency; single responsibility; duplicate logic; branch complexity; meaningful comments; input validation; exceptions; resource closure; access control; tests; performance; API contracts.

Documentation: review record, files/lines reviewed, attendees, checklist, defect ID, description, severity, owner, due date, disposition and closure evidence. Follow-up: author fixes accepted issues, reviewer verifies each fix, reruns relevant tests/static analysis, checks for regressions, and moderator closes only when exit criteria are met.

4. Peer review vs formal review; e-commerce critical module

A peer review is generally a developer-to-developer review that may be lightweight and flexible. A formal review follows defined roles, entry/exit criteria, documented findings and formal follow-up. For a five-developer e-commerce team, select the checkout/payment module: author, two reviewers (business logic and security), moderator/team lead, and recorder (roles may overlap in a small team).

Procedure: author opens a small reviewable change and supplies requirements/tests; reviewers inspect independently; check totals, discounts, inventory locking, payment idempotency, authorization, validation and error paths; record each defect with line, reproduction/reason, severity and suggested outcome; discuss unclear items respectfully; author fixes; reviewers re-check the diff and tests; merge only when critical/high defects are resolved and approvals/checks pass. Keep a review log and add regression tests for defects.

5. Coding-review checklist (25 criteria)

Use this checklist during peer review; mark each item pass/fail/not applicable and record evidence.

Criteria and reasons:
1. Naming: Use meaningful, consistent names and language conventions. Why: Improves readability and reduces misunderstanding.
2. Single responsibility: Keep methods/classes focused. Why: Simplifies testing and change.
3. Small methods: Avoid overly long methods. Why: Makes logic easier to review.
4. No duplication: Extract repeated logic appropriately. Why: Prevents inconsistent fixes.
5. Formatting: Follow consistent indentation and braces. Why: Reduces visual errors.
6. Constants: Replace unexplained magic numbers/strings. Why: Clarifies intent and enables safe changes.
7. Comments: Explain why/constraints, not obvious syntax. Why: Preserves useful context.
8. Input validation: Validate type, range, length and format. Why: Stops invalid data entering business logic.
9. Output encoding: Encode untrusted data for its output context. Why: Reduces injection risks.
10. Authentication: Verify identity securely. Why: Prevents unauthorized access.
11. Authorization: Check permission on every protected operation. Why: Avoids privilege escalation.
12. Secrets: Never hard-code keys/passwords. Why: Reduces credential leakage.
13. SQL safety: Use parameterized queries. Why: Prevents SQL injection.
14. Error handling: Handle expected errors explicitly. Why: Supports recovery and clear messages.
15. Exceptions: Do not swallow exceptions; preserve useful context. Why: Avoids hidden failures.
16. Logging: Log actionable events without secrets/PII. Why: Helps diagnosis while protecting privacy.
17. Resource management: Close files, streams, DB connections; use try-with-resources where applicable. Why: Prevents leaks.
18. Concurrency: Protect shared mutable state and define lock strategy. Why: Prevents races/deadlocks.
19. Performance: Avoid needless loops/queries and measure hot paths. Why: Improves responsiveness.
20. Algorithms: Choose suitable data structures/complexity. Why: Scales better.
21. Testing: Add unit, integration and negative tests. Why: Catches regressions.
22. Boundary cases: Test empty, min/max and overflow cases. Why: Finds edge defects.
23. Dependencies: Use supported, reviewed versions. Why: Reduces known vulnerabilities.
24. Maintainability: Prefer clear abstractions over clever code. Why: Lowers future change cost.
25. Accessibility/API: Follow API contracts and accessible UI semantics. Why: Improves interoperability and inclusion.

6. Equivalence Partitioning: Student Result Management System

Each of five subject marks should be a numeric value from 0 through 100 inclusive. Valid partition: 0–100. Invalid partitions: below 0; above 100; non-numeric text; blank/missing; decimal if only integers are permitted; malformed input. Apply each partition to each subject or use representative subjects plus data-driven coverage.

Representative tests: [50,60,70,80,90] → total 350, percentage 70%, grade according to specified grading policy, pass/fail according to pass threshold. [0,0,0,0,0] → total 0, percentage 0%; [100,100,100,100,100] → total 500, percentage 100%. A mark -1 or 101 should be rejected; “abc” and blank should show validation errors. Test one subject invalid at a time and multiple invalid fields together. The prompt does not specify grade bands or per-subject pass marks, so those expected grade/pass outcomes must be defined by requirements before finalizing tests.

7. Shopping discounts: Equivalence Partitioning and BVA

Classes: purchase <■1,000 = 0%; ■1,000–■4,999 = 5%; ■5,000–■9,999 = 10%; ■10,000–■49,999 = 15%; ≥■50,000 = 20%. Test representative values ■500, ■2,000, ■7,000, ■20,000, ■60,000.

Boundary tests (assuming whole-rupee amounts): ■999/■1,000/■1,001; ■4,998/■4,999/■5,000/■5,001; ■9,998/■9,999/■10,000/■10,001; ■49,998/■49,999/■50,000/■50,001. Expected discount is the rate for the class containing the purchase amount. Also test zero and negative totals, fractional currency if allowed, overflow, and cart total

after returns. Likely defects: < vs ≤ mistakes, gaps/overlaps, wrong rate, rounding errors, applying discount before/after tax incorrectly, or integer overflow.

8. Test to Pass and Test to Fail: ATM

Test to Pass confirms valid use: valid card/PIN, active account, sufficient funds, allowed amount, network available; verify cash dispensed, account debited once, receipt/history updated. Test exact daily/transaction limit where permitted.

Test to Fail deliberately supplies invalid/unfavourable conditions: wrong PIN; blocked/expired card; zero, negative or over-limit amount; insufficient balance; network unavailable; host timeout after request; repeated button press/retry. Verify no unauthorized cash, no duplicate debit, clear safe message, audit event, and correct reversal/reconciliation after uncertain timeout. For timeout, system must not assume failure if the bank may have processed it; use transaction reference/idempotency and reconciliation. Test recovery when network returns.

9. Data testing: hospital appointments

Create a data matrix: valid age and appointment; invalid negative/implausibly high age; boundary ages per policy; missing age/doctor/date/payment; wrong types (letters in age, malformed date); duplicate booking/payment request; inconsistent data (appointment before date of birth or doctor unavailable); extreme long strings/numeric overflow; malicious input such as script/SQL-like text; invalid payment status; timezone/date boundary.

Check client-side and server-side validation, schema constraints, referential integrity, normalization, uniqueness/idempotency, access control, escaping, privacy, audit trails, backup/restore and retention. Expected outcomes should be deterministic: accept valid records; reject or safely normalize invalid ones; never create duplicate appointments/charges; avoid exposing stack traces or sensitive patient details. Use synthetic test data, not real patient records.

10. State Transition Testing: bank account

Model states: New Account → Active → Suspended → Active → Closed. Define guards and actions: New→Active after verification; Active→Suspended after suspicious activity; Suspended→Active after authorized review; Active→Closed after closure requirements; possibly Suspended→Closed only if policy explicitly permits. Closed is terminal.

Valid tests: activate verified new account; suspend active account; reactivate suspended account after clearance; close eligible active account; verify closed account cannot transact. Invalid tests: New→Closed without required steps; Closed→Active; Closed→Suspended; transaction while suspended/closed; repeated activation. For every transition verify state, permissions, notifications, audit log, balance handling and retry/idempotency. Because the prompt omits exact closure rules, treat them as requirements to clarify.

11. State-based testing: food delivery

Happy path: Order Placed→Payment Confirmed→Restaurant Accepted→Food Preparing→Out for Delivery→Delivered. Alternate paths: payment attempt→Payment Failed; eligible cancellation→Cancelled; successful payment followed by cancellation/refund policy→Refund Initiated. Define exactly when cancellation is permitted and how refund completion is represented.

Test invalid skips (Placed→Delivered), duplicate payment confirmation, repeated restaurant acceptance, cancellation after dispatch, late/out-of-order events, duplicate webhook, driver status before assignment, timeout and retry, app/server restart, payment success arriving after a timeout, and refund provider failure. Verify idempotent event handling, inventory/order status consistency, customer notifications, audit trail, and recovery/reconciliation.

12. Decision Table: insurance claim

Conditions: policy valid (Y/N), premium paid (Y/N), accident covered (Y/N), documents complete (Y/N), claim amount within policy limit (Y/N). Example action rule: approve only when all five are Y; otherwise reject or refer for correction according to policy. A more realistic system may distinguish “pending documents” from “rejected”; define that before testing.

Compact decision table (each column is a rule):
R1: Y,Y,Y,Y,Y → Approve
R2: N,–,–,–,– → Reject: policy invalid
R3: Y,N,–,–,– → Reject/hold: premium unpaid
R4: Y,Y,N,–,– → Reject: not covered
R5: Y,Y,Y,N,– → Pending: documents missing
R6: Y,Y,Y,Y,N → Refer/reject: over limit
Derive one test per rule, plus

amount-at-limit and just-over-limit boundary tests. “–” means the condition need not be evaluated for that rule.

13. Decision Table: login system

Conditions: registered username, correct password, account active, account unlocked, CAPTCHA valid. Access is granted only if all are true. A minimal effective suite includes one all-valid test and one test for each single failed condition, giving six tests; also test combinations of multiple failures to ensure the system fails safely.

Tests: (Y,Y,Y,Y,Y)→login succeeds; (N,–,–,–,–)→deny; (Y,N,–,–,–)→deny; (Y,Y,N,–,–)→deny; (Y,Y,Y,N,–)→deny; (Y,Y,Y,Y,N)→deny. “–” means the other condition is not needed to explain the expected denial. Verify generic error messages do not reveal whether a username exists, plus rate limiting, audit logging and safe session handling.

14. Black-box password test suite

Rules: length 8–20 inclusive; at least one uppercase, lowercase, digit and special character. Valid example: Abcdef1! (8 chars). Length boundaries: 7 chars invalid, 8 valid if composition passes, 9 valid; 19 valid, 20 valid, 21 invalid. Composition negatives: missing uppercase; missing lowercase; missing digit; missing special; all missing except one category; whitespace-only; empty; very long input.

Use one-factor-at-a-time tests to isolate defects, plus combinations for interaction. Confirm accepted passwords meet every rule and invalid ones are rejected with helpful messages. Clarify whether Unicode letters, spaces, and which special characters are allowed. EP covers valid/invalid classes; BVA detects off-by-one length checks; negative tests detect missing validation, crashes and unsafe error handling.

15. Static black-box testing of online shopping requirements

Static black-box testing reviews requirements/specifications without running code. It checks testability, clarity, completeness, consistency, feasibility and traceability. Example requirements to flag: “page should load quickly” (ambiguous/unverifiable without a time target); “free delivery for orders above ■500” vs “delivery fee applies to every order” (contradictory); no definition of whether ■500 qualifies (incomplete boundary); no stated refund window or payment-failure behavior (missing); “support all payment methods” (unverifiable unless methods are listed).

Checklist: Is each requirement uniquely identified? Is wording measurable and unambiguous? Are all user roles and permissions defined? Are normal, error, cancellation and timeout flows specified? Are currency, taxes, discounts and rounding defined? Are boundaries explicit? Are security/privacy/accessibility requirements stated? Are response-time and load targets measurable? Are integrations and failure handling described? Are audit, refund, inventory and duplicate-payment rules defined? Can a test case prove pass/fail? Are requirements consistent and traceable to business goals? Record questions and resolve them with the product owner before implementation.